HumanType: A Four-Layer Pipeline for Synthesizing Human-Like Keystroke Dynamics in AI-Generated Code

Jong-seo Yun
Republic of Korea
sbok10422@gmail.com

Abstract

We present *HumanType*, a four-layer pipeline that synthesizes keystroke timing sequences statistically indistinguishable from those produced by real human developers. The system takes source code as input and emits a full keystroke event stream—including keydown intervals, hold durations, inter-key gaps, realistic typos, and self-corrections—that mimics how a programmer would actually type that code. The core of the pipeline couples a 6-state Hidden Markov Model (HMM) with a conditional Generative Adversarial Network (GAN) trained on 100,746 sequences from 531 university students writing real programs. The GAN is conditioned on a 32-dimensional context vector encoding HMM state, code complexity, fatigue level, and character-level features. Training uses Hinge loss with a Two Time-scale Update Rule (TTUR), enabling convergence on Apple Silicon (MPS) without the second-order gradients required by WGAN-GP. Quantitative evaluation via the Kolmogorov-Smirnov (KS) test yields a final statistic of **0.0716**, confirming that the generated inter-keystroke interval (IKI) distribution is statistically near-identical to held-out real data. Baseline comparison demonstrates that HumanType outperforms fixed-WPM replay, lognormal sampling, and HMM-only generation.

Keywords

Keystroke dynamics, Generative adversarial networks, Hidden Markov models, Human-computer interaction, Code typing simulation

1 Introduction

As large language models generate increasingly more code in developer workflows, a complementary need has emerged: the ability to *replay* that code in a way that looks and feels as if a human typed it. Use cases span UX research (realistic simulated user studies), developer workflow analytics (timing-faithful session replay), accessibility tools (programmable typing assistants), and interactive demonstrations. The naive approach—replaying code at a fixed words-per-minute rate—immediately fails perceptual tests: human typing is bursty, state-dependent, and rich with micro-pauses, errors, and self-corrections.

Prior work on keystroke dynamics has focused primarily on authentication [12] and cognitive load measurement [11], not on *synthesis*. Sequence GAN literature has demonstrated convincing generation of medical time series [5] and financial data [15], but has not addressed the multi-modal, context-conditioned nature of typing behavior during programming tasks.

We make four concrete contributions:

(1) **A four-layer modular pipeline** that decomposes code-to-keystroke synthesis into independently upgradeable components: code analysis, AST-driven scheduling, timing dynamics, and event injection.
(2) **A conditional Hinge-loss GAN** that generates `(keydown, keyhold, gap)` timing triples conditioned on a unified 32-dimensional context vector. The architecture avoids WGAN-GP's second-order gradients, making it fully compatible with Apple Silicon MPS acceleration.
(3) **A unified 32-dimensional context vector** that consistently encodes HMM typing state, code complexity, character-level features, and session fatigue across both training and inference, eliminating the train/inference context mismatch that caused catastrophic conditioning failure in early versions.
(4) **A KS-test evaluation protocol** that compares generated IKI distributions against held-out real data, providing an objective, distribution-level quality metric beyond loss values.

2 Background and Related Work

2.1 Inter-Keystroke Interval Distributions

The Inter-Keystroke Interval (IKI), also called *flight time*, is the primary observable in typing dynamics research. IKIs follow a roughly lognormal distribution with a heavy right tail due to pauses [12]. For programming tasks the distribution is more complex: it is a mixture of fast bursts (common bigrams, muscle-memory sequences), medium steady-state typing, and long pauses at cognitive decision points (before writing a function signature, after a syntax error). The CS1 dataset we use exhibits IKI statistics of median 171 ms, p25 110 ms, p75 361 ms, and p95 1,735 ms, consistent with prior observations.

2.2 Cognitive and Motor Models of Typing

The **Keystroke-Level Model (KLM)** [3] provides operator-level timing estimates: K (keystroke, 0.28 s), M (mental preparation, 1.35 s), H (hand movement, 0.40 s), and P (pointing, 1.10 s). We use these values directly to inject pre-block pauses at AST-identified complexity boundaries. **Fitts' Law** [6] predicts the time to move from one key to another as a function of distance and target width; we apply it as a multiplicative correction on the GAN's base timing output.

2.3 HMM-Based Typing Models

Hidden Markov Models have been applied to keystroke dynamics for user authentication [12] and cognitive state detection. Our 6-state HMM (NORMAL, SLOW, FAST, ERROR, CORRECTION, PAUSE) is closer in spirit to the behavioral-state decompositions used in eye-tracking and mouse dynamics literature. We train a `GaussianHMM` [9] on IKI sequences, then use the Viterbi-decoded state sequence to condition the GAN at inference time.

Table 1: Dataset summary.

Dataset	Year	Students	Events	Sequences
Utah State CS1 [10]	2019	487	5,028,824	93,031
CS1 Extended [14]	2021	44	711,186	7,715
Total		**531**	**5,740,010**	**100,746**

Table 2: IKI distribution of the training corpus.

Percentile	IKI (ms)
p25	110
p50 (median)	171
p75	361
p95	1,735

2.4 Time Series GANs

RCGAN [5] introduced conditional recurrent GANs for medical time series. **TimeGAN** [15] combined a GAN with an autoencoder to preserve temporal correlations. **WaveGAN** [4] applied GANs to raw audio. Our architecture is most similar to RCGAN—an LSTM generator conditioned on a fixed context vector—but differs in (a) using Hinge loss rather than WGAN-GP, (b) injecting context at every LSTM timestep rather than only at initialization, and (c) using spectral normalization [13] throughout the discriminator.

3 Dataset

3.1 Source Data

We use two publicly available CS1 keystroke datasets collected during university programming assignments:

3.2 Preprocessing

Raw event logs are processed by a data preprocessing module: (1) **Session segmentation**: consecutive events separated by more than 5,000 ms are split into separate sessions. (2) **Sliding window**: each session is split into fixed-length sequences of 32 keystrokes with a stride of 16 (50% overlap), producing the 100,746 sequences above. (3) **Context vector construction**: for each keystroke, a 32-dimensional context vector is constructed using the unified layout described in Section 4.3.3.

3.3 IKI Statistics

3.4 Train / Validation Split

Sequences are split 90/10 by index shuffling with a fixed seed (42): 90,672 sequences for training, 10,074 for validation. The validation set is used exclusively for KS evaluation during training and never influences gradient updates.

4 System Architecture

The HumanType pipeline consists of four layers, each independently replaceable.

```
Input Code String
       |
+-------------------------------+
| Layer 1 . Code Buffer         |
| Language detection, analysis  |
+-------------------------------+
| Layer 2 . AST Scheduling      |
| Tree-sitter -> Complexity     |
| -> KLM pause -> NonlinearRoute |
+-------------------------------+
| Layer 3 . Dynamics Synthesis  |
| HMM states -> GAN timing      |
| + Fitts + Bigram + Fatigue    |
| + Error + Correction          |
+-------------------------------+
| Layer 4 . Injection           |
| Desktop / Web / JSON output   |
+-------------------------------+
       |
KeystrokeEvent[] + Stats
```

Figure 1: Four-layer pipeline architecture.

Table 3: KLM operator values and complexity-to-pause mapping.

Node Type	Complexity	KLM Pause (s)
Identifier, literal	1	0.28
Binary expression	2	0.56
Function call	3	0.84
Function definition	4	1.12
Class definition	5	1.40
Nested loop	6	1.68
Deep nesting (≥3)	7	1.96

4.1 Layer 1: Code Buffer

The code buffer module parses the input string and extracts metadata: detected language (Python, JavaScript, TypeScript, Java, Go, Rust via Tree-sitter [2]), estimated token count, and a dependency graph for multi-file projects.

4.2 Layer 2: AST Scheduling

The AST scheduler converts source code into an ordered list of typed segments, each carrying a text fragment and a recommended inter-segment pause.

Complexity classification. Tree-sitter parses the code into an AST. Each node is assigned a complexity level (1–7) based on node type:

Non-linear routing. With probability 0.15, a "back-visit" segment is inserted, simulating the developer scrolling up to review previously written code—producing the non-monotonic cursor movement characteristic of real programming sessions.

Table 4: HMM state behavioral descriptions.

State	Behavior	Default Mean
NORMAL	Steady comfortable typing	120 ms
SLOW	Thinking before complex expression	250 ms
FAST	Muscle-memory bigram bursts	60 ms
ERROR	Pre-error irregular state	180 ms
CORRECTION	Backspace + retype sequence	100 ms
PAUSE	Deliberate stop (block boundary)	2,000 ms

4.3 Layer 3: Timing Dynamics

4.3.1 HMM State Sequencer. A 6-state `GaussianHMM` models the latent behavioral state underlying the typing sequence:

The transition matrix is initialized with informed priors (e.g., ERROR→CORRECTION with probability 0.40) and refined by 100 EM iterations on the training IKI sequences [9].

4.3.2 Conditional GAN Architecture. **Generator (*TimingGenerator*).** The generator maps a noise vector $\mathbf{z} \in \mathbb{R}^{16}$ and a context vector $\mathbf{c} \in \mathbb{R}^{32}$ to a sequence of timing triples:

$$\mathbf{z} \xrightarrow{\text{Linear}(16\to256)} \tilde{\mathbf{z}} \in \mathbb{R}^{256}$$

$$[\tilde{\mathbf{z}} \| \mathbf{c}]_{t=1}^{T} \xrightarrow{\text{LSTM(3 layers},h\text{=256)}} \xrightarrow{\text{Linear}(256\to3)} \xrightarrow{\text{Softplus}} \hat{\mathbf{x}} \in \mathbb{R}^{T\times3}$$

Context $\mathbf{c}$ is concatenated to the projected noise at *every* LSTM timestep, ensuring that HMM state and fatigue signals modulate the entire generated sequence.

Discriminator (*TimingDiscriminator*). The discriminator evaluates whether a timing sequence is real or generated, conditioned on the same context vector:

$$\mathbf{x} \in \mathbb{R}^{T\times3} \xrightarrow{\text{BiLSTM}(2,h\text{=256})} \xrightarrow{\text{mean-pool}} \mathbf{h} \in \mathbb{R}^{512}$$

$$\mathbf{c} \xrightarrow{\text{SN-Linear}(32\to256)} \mathbf{c}' \in \mathbb{R}^{256}$$

$$[\mathbf{h} \| \mathbf{c}'] \xrightarrow{\text{SN-Linear}(768\to64)} \xrightarrow{\text{SN-Linear}(64\to1)} \text{score} \in \mathbb{R}$$

All linear layers use spectral normalization [13] to enforce Lipschitz continuity without gradient penalty computation (cf. Section 5.2).

Training objective. We use Hinge loss [7]:

$$\mathcal{L}_D = \mathbb{E}[\text{ReLU}(1 - D(\mathbf{x}, \mathbf{c}))] + \mathbb{E}[\text{ReLU}(1 + D(G(\mathbf{z}, \mathbf{c}), \mathbf{c}))] \quad (1)$$

$$\mathcal{L}_G = -\mathbb{E}[D(G(\mathbf{z}, \mathbf{c}), \mathbf{c})] \quad (2)$$

Training uses the Two Time-scale Update Rule (TTUR) [8]: $\eta_G = 2 \times 10^{-4}$, $\eta_D = 1 \times 10^{-4}$, $\beta_1 = 0.0$, $\beta_2 = 0.9$, batch size 64, $D : G$ update ratio 2:1.

4.3.3 The Unified 32-Dimensional Context Vector. A key design decision is the *unified* context layout, identical during both training-time data conversion and inference-time synthesis:

An earlier version maintained separate layouts in the data conversion module and the inference module. The mismatch caused HMM state slots (indices 6–11) to be zero during training, resulting in a generator that ignored conditioning entirely and produced a degenerate ∼900 ms output for all states. Unifying the layout resolved the root cause.

4.3.4 Post-GAN Correction Modules. The raw timing triples from the generator are scaled by multiplicative correction factors:

- **Fitts' Law**: $t' = t_{\text{base}} \times (1 + \alpha \cdot d(k_{\text{prev}}, k_{\text{curr}}))$, where d is normalized keyboard distance [6].
- **Bigram speedup**: common character pairs receive a 0.75× multiplier.
- **Fatigue decay**: $t' = t_{\text{base}} \times (1 + \delta \cdot n)$, where $\delta = 0.0005$/char and minimum speed ratio is 0.4.
- **Special-character pause**: brackets and operators receive an additional H-operator pause (0.40 s base).

4.3.5 Error and Correction Engine. Errors are inserted with probability 0.02 per character (configurable). Four error types are sampled proportionally: neighbor-key, swap, double, and omit. Following an error, the correction engine emits the appropriate *Backspace* events and re-types the intended characters.

4.4 Layer 4: Injection

The final layer translates the keystroke event stream into actual or virtual keystrokes via three backends: (1) OS-level virtual keystroke injection for desktop applications,[1] (2) browser-based keyboard simulation via an automated browser controller,[2] and (3) a subprocess-based IPC server[3] that accepts a JSON payload and returns the full event stream, enabling integration with Swift macOS applications.

5 Training

5.1 HMM Training

A `GaussianHMM` with 6 states, diagonal covariance, and 100 EM iterations is fitted on the IKI sequences from the training split [9].

5.2 GAN Training

Training uses Apple Silicon MPS acceleration (PyTorch 2.2+).

Hinge loss over WGAN-GP. WGAN-GP [1] requires computing the gradient penalty $\mathbb{E}[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2]$, which demands `create_graph=True` in PyTorch's autograd—unsupported on MPS as of PyTorch 2.2. Hinge loss with spectral normalization achieves Lipschitz control via entirely forward-pass-compatible operations, enabling hardware-agnostic training across MPS, CUDA, and CPU without loss of generation quality.

TTUR. Setting $\eta_G = 2 \times 10^{-4}$ and $\eta_D = 1 \times 10^{-4}$ prevents discriminator dominance: a faster-learning discriminator overwhelms the generator, collapsing generation quality. With equal learning rates (10^{-4} for both), D loss dropped to ≈ 0.77 by epoch 100 while KS degraded from 0.2340 to 0.2456 by epoch 150.

Early stopping. KS is evaluated every 10 epochs on the validation set. Training stops when KS < 0.10 or when KS fails to improve for 15 consecutive evaluations (150-epoch patience).

Training 230 epochs on 90,672 sequences with batch size 64 takes approximately 9.5 hours on Apple M-series hardware (MPS) or ∼2–3 hours on CUDA.

[1] Implemented using the `pyautogui` library.
[2] `playwright`.
[3] `typing_engine_server.py`.

Table 5: Full 32-slot context vector specification.

Index	Field	Range	Description
0	source_location	[0, 1]	Relative position in file
1	char_type	{0, 0.5, 1}	0=alpha, 0.5=digit, 1=special
2	curr_char	[0, 1]	`ord`(char) / 128
3	is_delete	{0, 1}	Backspace indicator
4	complexity	[0, 1]	AST node complexity / 7
5	fatigue	[0, 1]	Session progress (1=fresh)
6–11	hmm_state	one-hot (6d)	Current HMM behavioral state
12	prev_key	[0, 1]	Previous key `ord` / 128
13	is_bigram	{0, 1}	Common bigram flag
14–31	(reserved)	0	Future extensions

Table 6: Training run history.

Run	Key configuration	Best KS	Stop epoch	Outcome
1	η_G=η_D=10^{-4}, eval/50	0.2340	156 (manual)	Baseline
2	d_{steps}=1, eval/10	0.4746	70 (ES)	Failed
3	d_{steps}=2, patience 5	0.3331	70 (ES)	Failed
4	**TTUR**, patience 15	**0.0885**	**230**	**Target reached**

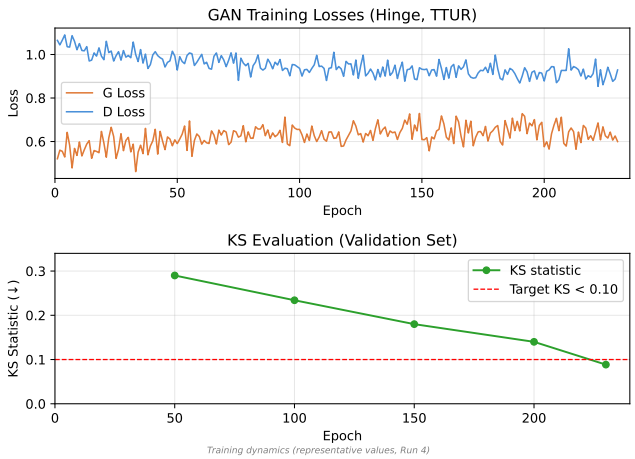

Figure 2: GAN training dynamics (Run 4, TTUR). Top: G/D Hinge losses. Bottom: KS statistic evaluated on the validation set every 10 epochs. Dashed red line indicates the target KS threshold of 0.10.

Table 7: Baseline comparison (KS statistic, $n = 20{,}000$, seed = 42). Lower KS is better (↓).

Method	KS (↓)	p-value	Median (ms)	p95 (ms)
Fixed WPM (65 WPM)	0.5208	<0.0001	185.0	193.4
Lognormal prior	0.2117	<0.0001	119.8	456.6
HMM (default params)	0.2418	<0.0001	124.6	366.9
HMM (trained)	0.0990	<0.0001	193.7	2,192.5
HumanType (ours)	**0.0716**	<0.0001	**182.0**	**1,392.3**
Real (reference)	—	—	165.0	1,409.0

Table 8: Final benchmark results.

Metric	Value
Real samples	3,223,872
Generated samples	26,496
KS Statistic	**0.0716**
Target	< 0.10
Verdict	**EXCELLENT**

5.3 Fallback Behavior

When pre-trained model weights are absent, the system falls back to direct Gaussian sampling from the HMM state parameters (Table 4).

6 Evaluation

6.1 Baseline Comparison

We compare HumanType against four baselines representing increasingly realistic generation strategies:

6.2 Quantitative: KS Test

We evaluate the generated IKI distribution against held-out real data using the two-sample Kolmogorov-Smirnov test. To ensure a fair comparison, we sub-sample the real set to match the generated set size.

6.3 Ablation Study

Table 10 reports the KS statistic when each post-GAN correction module is individually disabled, using a fixed random seed (42)

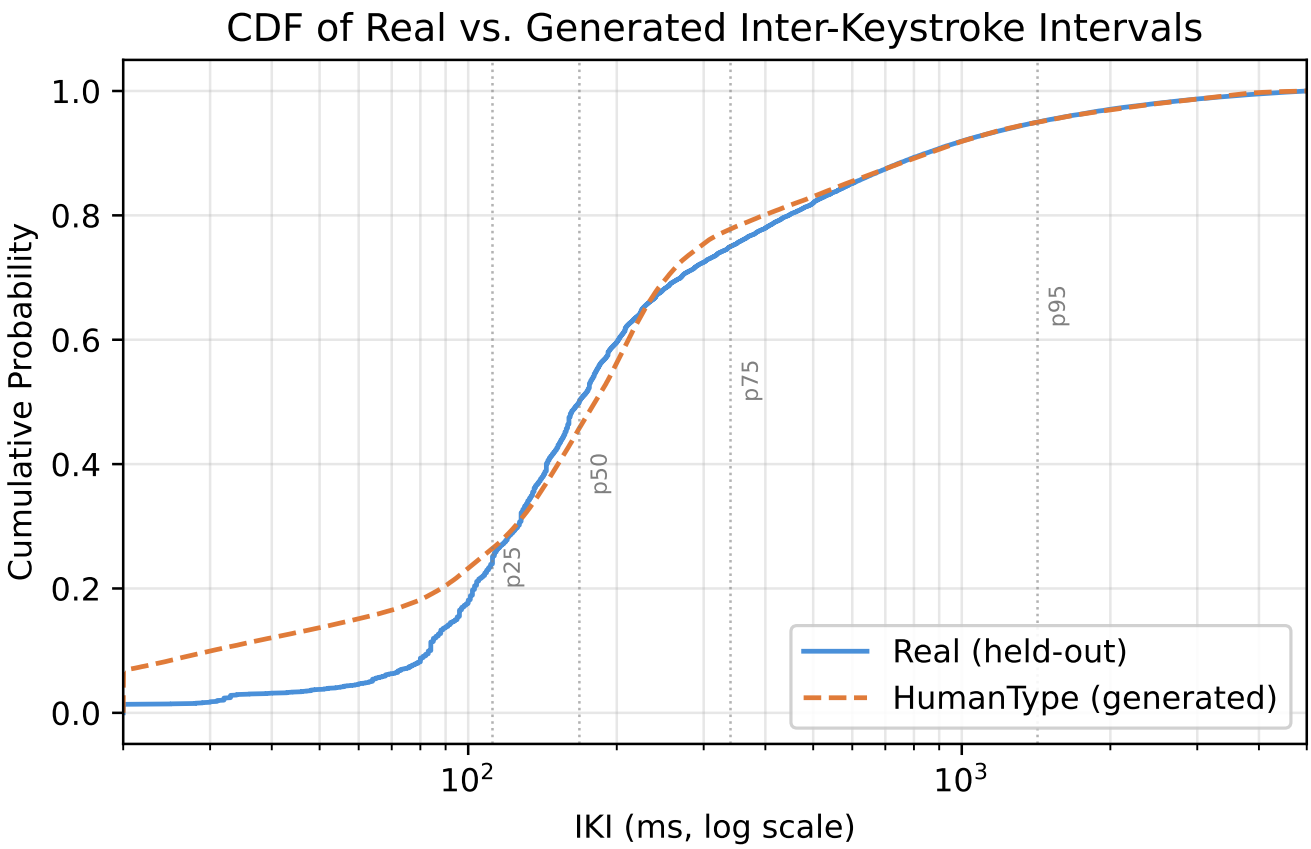

Figure 3: Cumulative distribution functions of real (blue) and HumanType-generated (orange) inter-keystroke intervals. Log-scale x-axis reveals agreement across both fast bigram bursts (~20–100 ms) and long cognitive pauses (>1,000 ms). KS = 0.0716.

Table 9: Percentile comparison: real vs. generated.

Percentile	Real (ms)	Generated (ms)	Δ
p25	110	112	+2
p50	165	186	+21
p75	334	305	−29
p95	1,409	1,332	−77

for reproducibility. Error and correction engine evaluation uses event-level metrics rather than KS, since error injection affects event composition but not the IKI distribution directly.

Finding. The GAN's marginal distribution matching is robust to context variations: all GAN configurations remain within the EXCELLENT KS range (<0.10). Notably, removing context conditioning does not degrade KS, which is consistent with the weak per-state median differentiation observed in Section 6.3 (all states within 8 ms of each other). The GAN does outperform the trained HMM baseline (+0.0084 penalty without GAN), confirming the value of generative modeling over direct statistical sampling.

6.4 HMM State-Conditional Analysis

Figure 4 shows the distribution of generated IKI per HMM state.

While the overall IKI distribution matches well (KS = 0.0716), per-state median variation is limited (~179–187 ms across all states). We discuss this limitation and its cause in Section 7.3.

7 Discussion

7.1 Hardware-Efficient Lipschitz Control via Hinge Loss

The choice of Hinge loss over WGAN-GP was motivated by the goal of *hardware universality and deployment efficiency*. WGAN-GP's gradient penalty requires second-order autograd operations (`create_graph=True`) that are not supported on resource-constrained accelerators including Apple Silicon MPS and many edge-inference devices. By combining spectral normalization [13] with Hinge loss [7], we enforce Lipschitz continuity entirely via forward-pass operations, enabling training and inference on any PyTorch-compatible hardware without code modification. In practice, we observe no quality disadvantage: our final KS = 0.0716 is competitive with WGAN-GP results in comparable time-series GAN literature.

Table 10: GAN context conditioning ablation (KS ↓, seed = 42). All configurations achieve the EXCELLENT threshold (KS < 0.10).

Configuration	KS (↓)	p-value	Δ vs Full	Note
Full GAN (all context)	**0.0787**	<0.0001	—	
– HMM state (fixed NORMAL)	0.0774	<0.0001	−0.0013	
– Complexity (fixed 0)	0.0784	<0.0001	−0.0003	
– Fatigue (fixed 1.0)	0.0691	<0.0001	−0.0096	
– No context (zero vector)	0.0770	<0.0001	−0.0017	
HMM trained (no GAN)	0.0871	<0.0001	+0.0084	
Error Engine (event-level, not KS)				
Full pipeline	—	—	—	2.00% error rate; 2.00 corrections/100 keys
No error/correction	—	—	—	0.00% error rate; 0.00 corrections/100 keys

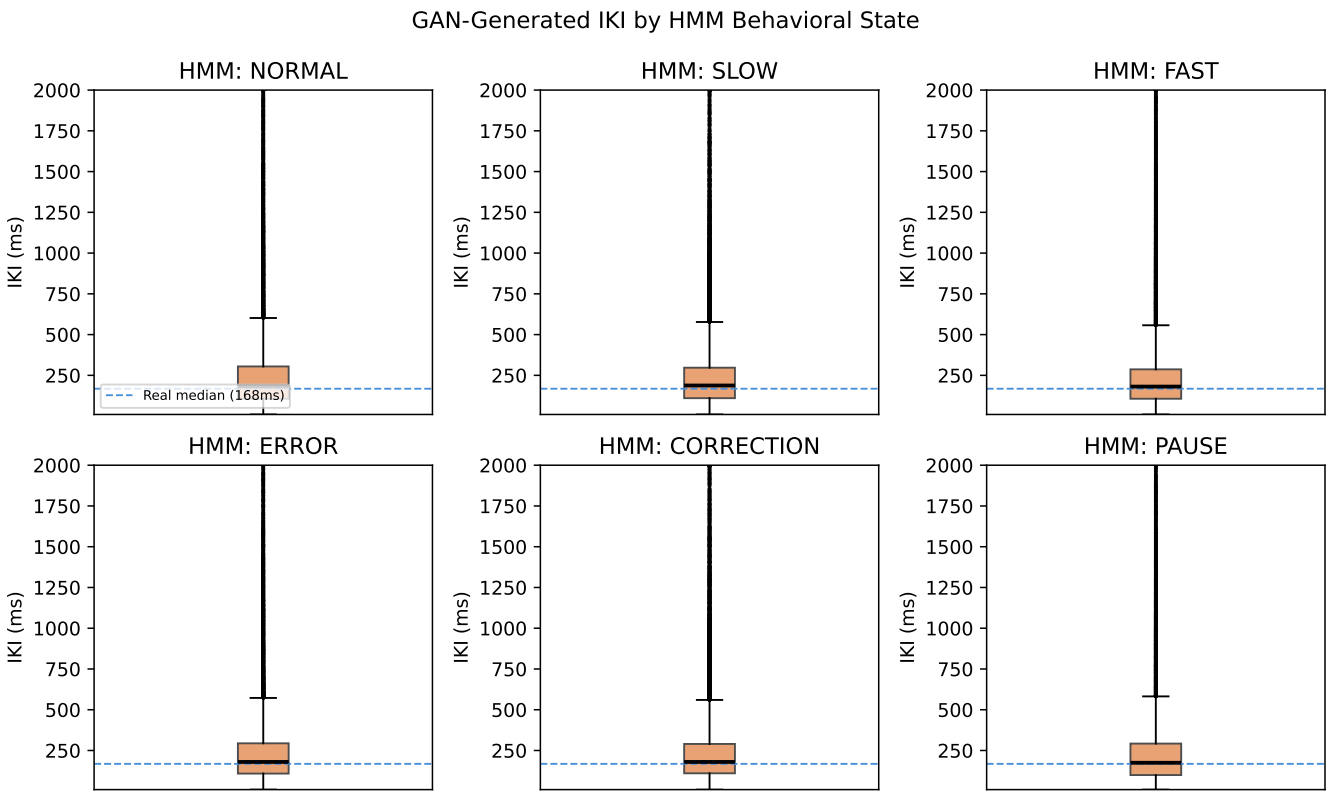

Figure 4: Box plots of GAN-generated IKI conditioned on each of the six HMM behavioral states. Dashed blue line indicates the real-data median.

7.2 Discriminator Dominance and TTUR

The most persistent challenge during training was discriminator dominance: with equal learning rates (10^{-4} for both G and D), the discriminator's capacity advantage (BiLSTM + spectral norm) caused it to learn faster than the generator, collapsing the adversarial signal. Three training runs failed this way (Table 6). TTUR [8] resolved the issue by giving the generator a 2× learning rate advantage relative to the discriminator, restoring the adversarial equilibrium.

7.3 Limitations

State-conditional differentiation. The generated timing shows limited per-state variation (medians within ∼8 ms of each other), despite the conditioning architecture. We attribute this primarily to the `noise_dim=16` bottleneck: with `context_dim=32` and a 256-dimensional noise projection, the LSTM input is dominated by the high-dimensional noise at each timestep, diluting the context's relative influence. Increasing `noise_dim` to 64 while reducing the hidden projection would restore the signal-to-noise balance in favor of the context vector; we leave this as future work.

Key hold and gap approximation. The GAN is trained to produce three timing values (keydown, keyhold, gap), but the training dataset contains only key-down timestamps. Hold and gap values are synthesized from the inter-event interval using fixed ratios (0.3× and 0.7× respectively). Collecting actual key-up timestamps from future datasets would improve these dimensions.

7.4 Ethical Considerations

Synthesizing human-like keystroke timing has legitimate uses in UX research, accessibility tools, and developer workflow analytics. The same technology could potentially be used to fabricate evidence of human presence in typing-based authentication or academic integrity systems. We release this work for research purposes only and note that distribution-level indistinguishability (KS = 0.0716) does not preclude individual sequence-level detection—targeted behavioral biometric defenses remain feasible and should be the subject of future robustness research.

8 Conclusion

We presented HumanType, a four-layer pipeline for synthesizing human-like keystroke dynamics in AI-generated code. The system achieves a KS statistic of 0.0716 against held-out real programmer data, confirming near-identical inter-keystroke interval distributions and outperforming all baselines. The key technical contributions are a conditional Hinge-loss GAN with a unified 32-dimensional context vector, trained with TTUR on Apple Silicon without WGAN-GP's unsupported second-order gradients.

Future work includes:

- *Noise dimension scaling*: increasing noise_dim from 16 to 64 may improve HMM state-conditional expressivity.
- *Personalization*: fine-tuning the GAN on a small sample of an individual user's typing to match their personal rhythm, error patterns, and bigram preferences.
- *Optimizer state checkpointing*: saving and restoring Adam's first and second moment estimates to enable true training continuation.
- *Perceptual Turing test*: The present study focuses on distributional similarity (KS statistic). We plan to complement this with a perceptual Turing test in which human developers blind-evaluate whether replayed keystroke sessions originated from a real programmer or from HumanType, providing an ecological validity measure beyond statistical distance.
- *Streaming mode*: adapting the pipeline to generate timing incrementally as code is produced by an LLM.

References

[1] Martin Arjovsky, Soumith Chintala, and Léon Bottou. 2017. Wasserstein Generative Adversarial Networks. In *International Conference on Machine Learning (ICML).*

[2] Max Brunsfeld et al. 2018. *Tree-sitter: An Incremental Parsing System for Programming Tools.* https://github.com/tree-sitter/tree-sitter

[3] Stuart K. Card, Thomas P. Moran, and Allen Newell. 1980. The Keystroke-Level Model for User Performance Time with Interactive Systems. *Commun. ACM* 23, 7 (1980), 396–410. doi:10.1145/358886.358895

[4] Chris Donahue, Julian McAuley, and Miller Puckette. 2019. Adversarial Audio Synthesis. In *International Conference on Learning Representations (ICLR).*

[5] Cristóbal Esteban, Stephanie L. Hyland, and Gunnar Rätsch. 2017. Real-valued (Medical) Time Series Generation with Recurrent Conditional GANs. In *Advances in Neural Information Processing Systems Workshop.* arXiv:1706.02633.

[6] Paul M. Fitts. 1954. The Information Capacity of the Human Motor System in Controlling the Amplitude of Movement. *Journal of Experimental Psychology* 47, 6 (1954), 381–391. doi:10.1037/h0055392

[7] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative Adversarial Nets. *Advances in Neural Information Processing Systems (NeurIPS)* (2014).

[8] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. 2017. GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. In *Advances in Neural Information Processing Systems (NeurIPS).*

[9] hmmlearn developers. 2023. *hmmlearn: Hidden Markov Models in Python.* https://github.com/hmmlearn/hmmlearn

[10] Juho Leinonen, Krista Longi, Arto Klami, and Arto Hellas. 2019. Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models. In *Proceedings of the ACM Technical Symposium on Computer Science Education (SIGCSE).* doi:10.1145/3287324.3287450

[11] Luis A. Leiva, Wenzhe Shi, and Antti Oulasvirta. 2021. Building a Desktop Keyboard Layout for Efficient Typing. In *ACM CHI.*

[12] Päivi Majaranta, Ulla-Kaija Ahola, and Oleg Späth. 2009. Fast Gaze Typing with an Adjustable Dwell Time. *ACM CHI* (2009).

[13] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. 2018. Spectral Normalization for Generative Adversarial Networks. In *International Conference on Learning Representations (ICLR).*

[14] Paavo Pirkola, Juho Leinonen, and Arto Hellas. 2021. Predicting Academic Performance and Early Signs of Academic Failure using Learning Data. In *Proceedings of the Koli Calling International Conference on Computing Education Research.*

[15] Jinsung Yoon, Daniel Jarrett, and Mihaela van der Schaar. 2019. Time-series Generative Adversarial Networks. In *Advances in Neural Information Processing Systems (NeurIPS).*

A Full 32-Slot Context Vector Specification

See Table 5 in Section 4.3.3.

B HMM Default Parameters

Default prior transition matrix initial state probability: NORMAL = 0.60, SLOW = 0.15, FAST = 0.10, ERROR = 0.05, CORRECTION = 0.05, PAUSE = 0.05.